Why did the programmer quit their job?

A never got arrays!

CMPT 295

Unit - Machine-Level Programming

Lecture 13 – Assembly language – Program Control – Conditional Statements

Last lectures

1. **Absolute**
2. **Indirect**
3. **"Base + displacement"**
4. **2 indexed**
5. **4 scaled indexed**

- x86-64 instructions can have various types of operands
  - Immediate (constant integral value)
  - Register (16 registers)
  - Memory addressing mode (various forms)
    - General Syntax: **`Imm(r_b, r_i, s)`**
- **`leaq`** - load effective address instruction (also **`leal`**)
- Arithmetic & logical instructions
  - Arithmetic instructions: **`add*`, `sub*`, `imul*` `inc*`, `dec*`, `neg*`, `not*`**
  - Logical instructions: **`and*`, `or*`, `xor*`**
  - Shift instructions: **`sal*`, `sar*`, `shr*`**
- Demo
  - Observation: C compiler will figure out different instruction combinations to carry out the computations in our C code

* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

Review - Demo

arith.c

```c
long arith(long x, long y, long z) {
  long t1 = x + y;
  long t2 = z + t1;
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

- Observation: C compiler will figure out different instruction combinations to carry out the computations in our C code

arith_mine.s

```
    .globl  arith
arith:  # mine
    # x -> %rdi, y -> %rsi, z -> %rdx
    # addq %rsi, %rdi # Bad! because overwrites %rdi which is needed later on for "t3 = x + 4"
    # addq %rdi, %rsi # Bad! because overwrites %rsi which is needed later on for "t4 = y * 48"
    leaq (%rdi, %rsi), %rax  # t1 -> %rax <- function's returned value
    #OR      movq %rdi, %rax # x -> %rax
    #        addq %rsi, %rax # t1 = x + y; t1 -> %rax
    addq %rdx, %rax          # t2 = z + t1; t2 -> %rax
    addq $4, %rdi            # t3 = x + 4; t3 -> %rdi
    imulq $48, %rsi          # t4 = y * 48; t4 -> %rsi
    addq %rsi, %rdi          # t5 = t3 + t4; t5 -> %rdi
    imulq %rdi, %rax         # rval = t2 * t5; rval -> %rax
    ret
```

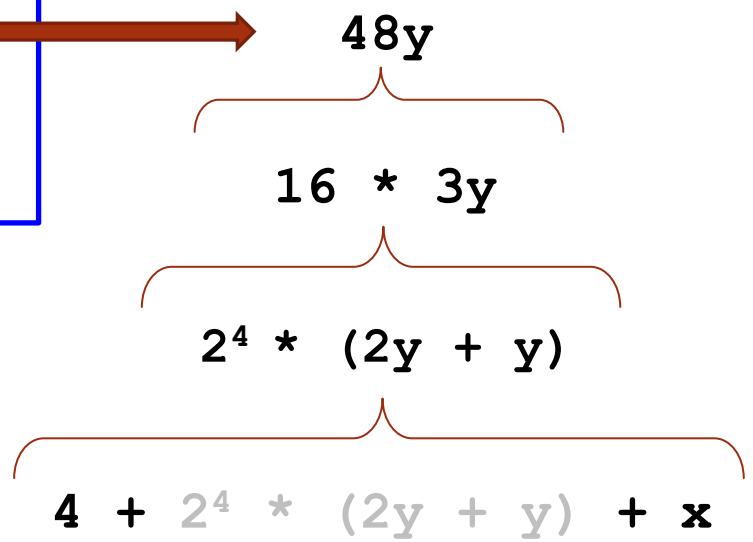
arith_gcc.s

```
arith:
.LFB0:
    .cfi_startproc
    endbr64
    movq    %rdx, %rax
    leaq    (%rsi,%rsi,2), %rcx
    salq    $4, %rcx
    leaq    4(%rdi,%rcx), %rdx
    addq    %rsi, %rdi
    addq    %rdi, %rax
    imulq   %rdx, %rax
    ret
```

Demo (cont'd)

arith.c

```
long arith(long x, long y, long z) {
  long t1 = x + y;
  long t2 = z + t1;
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```



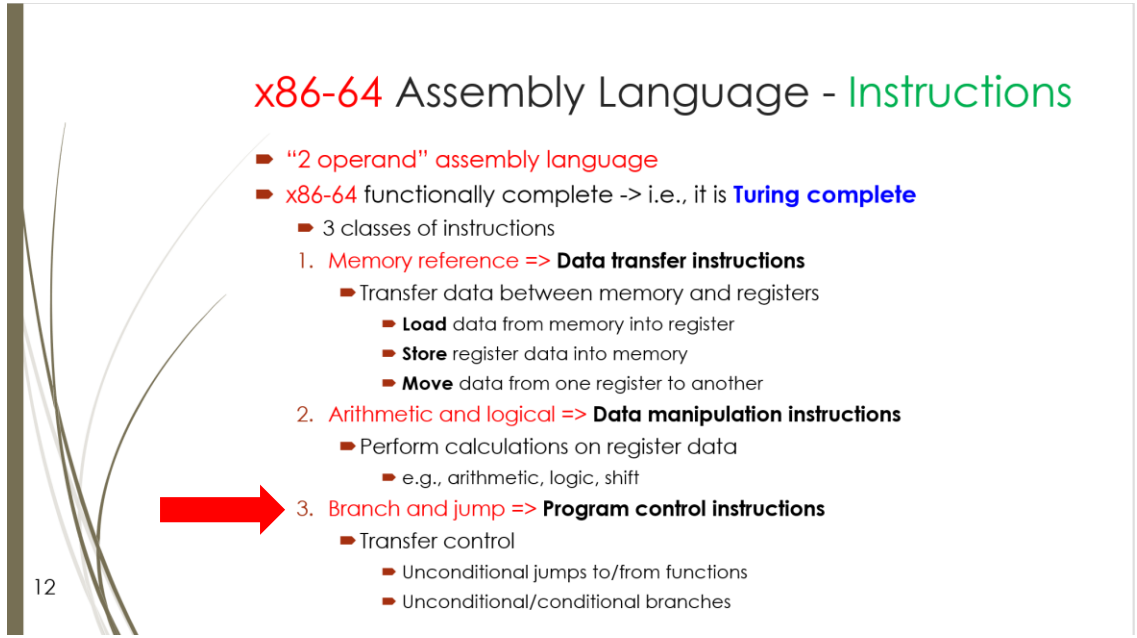
arith_gcc.s

```
arith:
.LFB0:
    .cfi_startproc
    endbr64
    movq    %rdx, %rax
    leaq    (%rsi,%rsi,2), %rcx
    salq    $4, %rcx
    leaq    4(%rdi,%rcx), %rdx
    addq    %rsi, %rdi
    addq    %rdi, %rax
    imulq   %rdx, %rax
    ret
```

Today's Menu

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Last class of instructions



- Remember Slide 12 from our Lecture 10:

  and the fact that x86-64 is a functionally complete assembly language

  i.e., that it is **Turing complete**
- We have covered the first and second classes of instructions:
  - Memory reference => **Data transfer instructions**
  - Arithmetic and logical => **Data manipulation instructions**
- Now, let's move on to
  - Branch and jump => **Program control instructions**

Program Control Overview

- We can control the execution flow of a program
  1. Based on a condition
  2. Unconditionally

---

in C

Control statements:

Conditional statements:
- **`if/else`**
- **`switch`**
- Conditional operator:

```
variable = condition ? expression1 : expression2;
```

Iterative statements:
- **`for`** loop
- **`while`** and **`do while`** loops

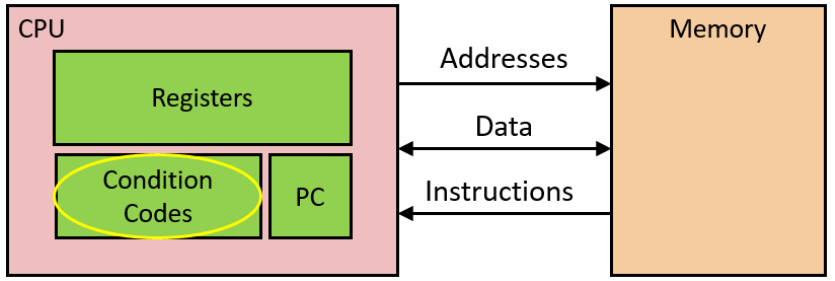
- function calls

in x86-64 assembly

Branching:
- **`cmp*`** instruction (***compare***)
- **`test*`** instruction (***test***)
- **`jx`** instructions (***jump***)
- **`cmov*`** instructions (***conditional move***)

→ **`call`** and **`ret`**

Remember from Lecture 10:



Condition Codes

- Condition codes are single bit registers

indicates whether carry out bit set
CF Carry Flag (for unsigned)

indicates whether result is 0
ZF Zero Flag

indicates whether result is –ve #
SF Sign Flag (for signed)

indicates whether result created +ve or –ve overflow
OF Overflow Flag (for signed)

- Instructions such as ***arithmetic*** and ***logical*** instructions, **`cmp*`** and **`test*`** set condition codes
- Instructions such as **`jX`** instructions use condition codes to decide whether to jump or not

Condition codes creating the …

Condition `X`	Condition	Description
`e`	`ZF`	Equal / Zero
`ne`	`~ZF`	Not Equal / Not Zero
`s`	`SF`	Negative
`ns`	`~SF`	Nonnegative
`g`	`~(SF^OF) &~ZF`	Greater (Signed)
`ge`	`~(SF^OF)`	Greater or Equal (Signed)
`l`	`(SF^OF)`	Less (Signed)
`le`	`(SF^OF) | ZF`	Less or Equal (Signed)
`a`	`~CF&~ZF`	Above (unsigned)
`b`	`CF`	Below (unsigned)

* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

`cmp*` and `test*` families of instructions

Syntax	Meaning/Effect	Condition	Example
`cmp* Src2, Src1`	`Src1 - Src2 ->`	`> 0? -> Src1 > Src2: g`	`cmpq %rsi,%rdi`
		`= 0? -> Src1 == Src2:e`	
		`< 0? -> Src1 < Src2: l`	

without saving the result in the destination operand (no **`Dest`**)

- Sets ***condition codes*** based on value of **`Src1 - Src2`**

Syntax	Meaning/Effect	Condition	Example

`test* Src2, Src1`	`Src1 & Src2`
	`Src1 & Src1 ->`
	Typically used when testing ***a register***
		`> 0? -> Src1 > 0: g`	`testq %rax,%rax`

		`= 0? -> Src1 = 0: e`	
		`< 0? -> Src1 < 0: l`	

without saving the result in the destination operand (no **`Dest`**)

- Sets ***condition codes*** based on value of **`Src1 & Src2`**
- Typically used when testing ***a register*** or when one of the operands is a ***bit mask***

Condition

jX jump family of instructions (branching)

- Jump to a different section of the assembly program depending on result of previous instruction (e.g., `add*`, `cmp*`, `test*`, ...), more specifically, based on the condition codes set by these instructions

Condition

jX	Description
je	Equal / Zero
jne	Not Equal / Not Zero
js	Negative
jns	Nonnegative
jg	Greater (Signed)
jge	Greater or Equal (Signed)
jl	Less (Signed)
jle	Less or Equal (Signed)
ja	Above (unsigned)
jb	Below (unsigned)
jmp	Unconditional

Label i.e., *memory address*

Example: `jge else`

Conditional statement: `if/else`

in C:

```
void func(long x, long y){
  if ( x < y ) {
    // stmts when true
  } else {
    // stmts when false
  }
  return;
}
```

in assembly:

```
func:
    # x in %rdi, y in %rsi
    cmpq %rsi,%rdi # x - y
    jge  else      #
    ...            # stmts when true
    jmp  endif     #
else:  ...         # stmts when false
endif: ret         #
```

label

label

`cmp* Src2, Src1`

Meaning/Effect

Src1 - Src2 -> > 0? -> Src1 > Src2: g
= 0? -> Src1 == Src2:e
< 0? -> Src1 < Src2: l

A **label** is a memory address

We branch (jump) when the condition is false

-> This technique is called *"coding the false condition first"*

Conditional statement: `if/else`

Test Case 1:
test data:
x = 5, y = 7
expected result:
`stmts when true`
executed

Test Case 2:
test data:
x = 7, y = 7
expected result:
`stmts when false`
executed

in C:

```
void func(long x, long y){
   if ( x < y ) {
      // stmts when true
   } else {
      // stmts when false
   }
   return;
}
```

in assembly:

```
func:
      # x in %rdi, y in %rsi
     cmpq %rsi,%rdi # x - y
     jge  else      #
     ...            # stmts when true
     jmp  endif     #
else:  ...          # stmts when false
endif: ret          #
```

`cmp* Src2, Src1`

Meaning/Effect

```
Src1 - Src2 -> > 0? -> Src1 > Src2: g
               = 0? -> Src1 == Src2:e
               < 0? -> Src1 < Src2: l
```

Example – `int abs(int x)`

Test Case 1:
test data:
x = 5
expected result:
5 is returned

Test Case 2:
test data:
x = -5
expected result:
5 is returned

in C:

```
int abs(int x){
   if ( x < 0 )
     x = -x;
   return x;
}
```

in assembly:

```
abs:
    # x in %edi, result in %eax
    movl %edi, %eax # eax <- x
                    #
                    # ret if x >= 0
                    # x = -x
endif:
    ret
```

Example – `int abs(int x)`

version 1 – using cmp* instruction

Test Case 1:
test data:
x = 5
expected result:
5 is returned

Test Case 2:
test data:
x = -5
expected result:
5 is returned

in C:

```
int abs(int x){
   if ( x < 0 )
     x = -x;
   return x;
}
```

in assembly:

```
abs:
    # x in %edi, result in %eax
    movl %edi, %eax # x -> %eax
                    #
                    # return x (if x >= 0)
                    # x = -x
endif:
    ret
```

Example – `int abs(int x)`

version 2 – using test* instruction

Test Case 1:
test data:
x = 5
expected result:
5 is returned

Test Case 2:
test data:
x = -5
expected result:
5 is returned

in C:

```
int abs(int x){
   if ( x < 0 )
     x = -x;
   return x;
}
```

in assembly:

```
abs:
    # x in %edi, result in %eax
    movl   %edi, %eax # eax <- x
    testl  %edi, %edi # if x < 0 then
    jns    endif      # return x
    negl   %eax       # x = -x
endif:
    ret               # return x
```

Example – `int abs(int x)`

Test Case 3:
test data:
x = 2147483647
expected result:
2147483647
is returned

Test Case 4:
test data:
x = -2147483648
expected result:
2147483648
is returned ?

in C:

```
int abs(int x){
   if ( x < 0 )
     x = -x;
   return x;
}
```

in assembly:

```
abs:
    # x in %edi, result in %eax
    movl   %edi, %eax # eax <- x
    cmpl   $0,   %edi # if x < 0 then
    jge    endif      # return x (if x >= 0)
    negl   %eax       # x = -x
endif:
    ret               # return x
```

`signed int` range: [-2147483648 .. 2147483647]

Example – `int abs(int x)`

Test Case 3:
test data:
x = 2147483647
expected result:
2147483647
is returned

Test Case 4:
test data:
x = -2147483648
expected result:
2147483648
is returned ?

in C:

```
int abs(int x){
   if ( x < 0 )
     x = -x;
   return x;
}
```

in assembly:

```
abs:
    # x in %edi, result in %eax
    movl %edi, %eax # eax <- x
                    #
                    # ret if x >= 0
                    # x = -x
endif:
    ret
```

`signed int` range: [-2147483648 .. 2147483647]

`int max(int x, int y)` – Homework

in C:

```
int max(int x, int y){
   int result = x;
   if (y > x)
      result = y;
   return result;
}
```

in assembly:

```
# x in %edi, y in %esi, result in %eax
max:
    movl %edi, %eax # result = x




endif:
    ret
```

Summary

- In C, we can control the execution flow of a program
  1. Conditionaly
     - Conditional statements: **`if/else`**, **`switch`** , …
     - Iterative statements: **`for`** loops, **`while`** loops, …
  2. Unconditionally
     - Functions calls
- In x86-64 assembly, we can also control the execution flow of a program
  - **`cmp*`** instruction (*compare*)
  - **`jX`** instructions (*jump*)
  - **`call`** and **`ret`** instructions

Next lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + **cmovX**
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations